

Skybox: Open-Source Graphic Rendering on Programmable RISC-V GPUs

Blaise Tine
Georgia Institute of Technology
USA
btine3@gatech.edu

Varun Saxena
Georgia Institute of Technology
USA
vsaxena36@gatech.edu

Santosh Srivatsan
Georgia Institute of Technology
USA
ssrivatsan30@gatech.edu

Joshua R. Simpson
California Polytechnic State
USA
simps03@calpoly.edu

Fadi Alzammar
California Polytechnic State
USA
falzamma@calpoly.edu

Liam Cooper
Georgia Institute of Technology
USA
lcooper43@gatech.edu

Hyesoon Kim
Georgia Institute of Technology
USA
hyesoon.kim@gatech.edu

ABSTRACT

Graphics rendering remains one of the most compute-intensive and memory-bound applications of GPUs and has been driving their push for performance and energy efficiency since its inception. However, the lack of open-source designs for the graphics rendering pipeline that runs modern GPU applications hinders GPU research. In this work, we present Skybox, a full-stack, open-source, 3D graphics framework with integrated hardware and software, including a compiler and simulation environment, that enables end-to-end GPU research. Using Skybox, we explore the design space of software versus hardware graphics rendering and propose a hybrid micro-architecture that accelerates the state-of-the-art Vulkan graphics API. Skybox also introduces novel compiler and system optimizations to support its unique RISC-V ISA baseline. We evaluated Skybox on high-end Altera FPGAs. We were able to synthesize and execute a 32-core (512 thread) Skybox graphics processor on the Altera Stratix 10 FPGA running at 230 Mhz, delivering a peak fill rate of 3.7 GPixels/s. Skybox is the first open-source, full-stack GPU software and hardware implementation that supports the Vulkan API.

CCS CONCEPTS

• **Hardware** → **Full-custom circuits**.

KEYWORDS

hardware accelerator, graphics, microarchitecture

ACM Reference Format:
Blaise Tine, Varun Saxena, Santosh Srivatsan, Joshua R. Simpson, Fadi Alzammar, Liam Cooper, and Hyesoon Kim. 2023. Skybox: Open-Source Graphic Rendering on Programmable RISC-V GPUs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '23), March 25–29, 2023, Vancouver, BC, Canada.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3582016.3582024



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '23, March 25–29, 2023, Vancouver, BC, Canada

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9918-0/23/03.
https://doi.org/10.1145/3582016.3582024

1 INTRODUCTION

In the past decade, graphics processing unit (GPU) research has mainly focused on improving the performance and efficiency of general-purpose computing on GPU (GPGPU) applications. However, graphics rendering was the main application target driving the inception of the GPU two decades ago and has since remained the driving force for technological innovations in GPU architectures. Two vectors have motivated the main changes in GPU architectures in the past decades: the demand for higher performance, driving the push for scalability with increased core counts [42] and energy efficiency [10]; and the demand for programmability, driving the push for programmable shaders [36] and the expansion of the graphics pipeline. The push for programmability has enabled GPUs to enter new markets in scientific computing, machine learning, graph analytics, and cryptocurrency.

Today's state-of-the-art GPUs [7] host thousands of general-purpose programmable cores to satisfy the diversity of workloads from various application domains. Although architecture research in academia has mainly focused on GPGPU cores, modern GPUs also implement optimized fixed-function hardware dedicated to graphics rendering acceleration which includes tessellation units, rasterizer units, texture units, render output units, etc [44]. These fixed-function unit designs are still a major design decision, as commercial GPUs are compared by the speed of rendering. Furthermore, the area dedicated to these graphics components occupies considerable die space which could have been allocated to the expansion of the general-purpose programmable pipeline, especially on mobile GPUs [53] [61] [69], because the core count is relatively small due to the limited power budget on these platforms. This puts into focus the cost of dedicated fixed-function graphics pipelines versus using that area for additional cores.

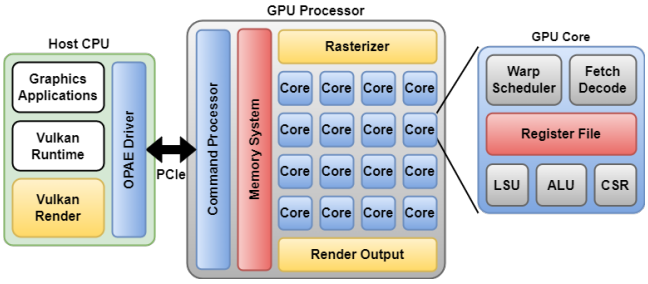


Figure 1: Skybox framework overview.

The need to study the cost of the fixed-function graphics pipeline is also driven by recent innovations in graphics rendering with ray tracing [68] [7] [34], which produces higher-quality results and is competing for the scarce area budget on embedded devices. Additionally, the current fixed-function graphics pipeline is suboptimal with respect to rendering non-traditional triangle primitives, such as Bézier curves, quads, lines, and points that are prevalent in vector graphics [9] [23] [46]. Those primitives are generally rendered in software on the CPU or on the GPU using the compute pipeline [11].

Studying the graphics pipeline design, especially the trade-off between fixed-function vs. programmable core, requires a detailed hardware design to model area, performance, and cost. Unfortunately, there has been no open-source hardware design to study the graphics rendering pipeline that can execute modern GPU workloads. Although recent work [47] [18] [40] has looked into this problem, the proposed solutions have focused on cycle-approximate simulation (CAS). CAS-based research has practical advantages, but simulating a complex microarchitecture at the IR level has several limitations that impact performance [20] and it is not equipped to study the design trade-off between fixed-function units and programmable cores.

To this end, we propose Skybox (shown in Figure 1), an open-source, full-stack GPU software and hardware framework that integrates a compiler and simulation environment needed for graphics hardware research. Skybox's hardware is implemented fully in RTL and synthesized on FPGA to enable real-time prototyping and evaluation of graphics workloads. Our work stems from the open-source Vortex [54] project that implements a full-scale GPGPU processor on FPGA. The Vortex microarchitecture lacks the graphics pipeline needed to accelerate 3D rendering applications.

Skybox implements the modern Vulkan graphics API for the first time on open-source hardware. Existing GPU simulators support OpenGL [15], an older API which modern applications often no longer support.

Another important point is that Skybox is based on the RISC-V ISA, which is driving the open hardware/software [17] effort. Nonetheless, there has been no open-source design that provides a graphics rendering pipeline. Currently, only a commercial design of a RISC-V GPU has been under development [66]. Extending Vulkan software for the 32-bit RISC-V ISA presented several challenges at both the system and the compiler levels. For example, Vulkan software increases the branch divergence and register pressure. Hence, to support Vulkan shaders, we devised a new hybrid stack extension scheme to expand the stack space of the GPU, and extended the LLVM compiler to automate split-join insertion for divergent branches.

Using Skybox, we can explore the design space of software implementation vs. fixed-function units. This trade-off is particularly important for RISC-V ISA-based systems, as RISC-V has a small number of instruction operands and a limited number of registers. Skybox implements a hybrid GPU architecture where the graphics rendering workload is distributed between the fixed-function and the programmable shader pipeline. More explicitly, Skybox keeps the front-end of the graphics pipeline, the vertex computation, and primitive setup on the programmable shader cores together with the fragment shader and moves the back-end rasterization, interpolation, depth stencil, and blender output units to fixed-function hardware to provide high-speed rendering throughput at a lower area cost.

This paper makes the following key contributions:

- We developed the first open-source hardware implementation and simulation framework of a RISC-V-based 3D graphics rendering accelerator that supports the Vulkan API. We also extended the Vulkan software stack to support our open-source GPUs.
- We explore the design trade-off for graphics acceleration to find the software and hardware boundaries and propose a new hybrid GPU architecture.
- We introduce a RISC-V ISA extension for accelerating graphics rendering and describe its implementation.
- We propose an implementation of a compiler-driven control-flow divergence transformation based on the RISC-V ISA to handle rendering loops inside graphics kernels.
- We propose a hybrid stack management architecture that extends the stack space of the GPU to support graphics workloads.

2 BACKGROUND ON GRAPHICS

2.1 The 3D Graphics Pipeline

2.1.1 Overview of the 3D graphics Pipeline. [1] Figure 3 illustrates the main stages of the traditional programmable 3D-graphics pipeline: the geometry processing stage in the front-end, and the rasterization and fragment processing stages in the back-end.

Geometry Processing Stage: The incoming vertices of a 3D model from a graphics application are transformed into triangle primitives and projected to 2D screen space.

Rasterization Stage: Generated triangle primitives are rasterized into the destination render target, generating pixels covered by each triangle primitive.

Fragment processing Stage: Per-pixel computation is used to generate the final color that is sent to the display.

2.1.2 Tile-based Rendering. TBR [6] [41] is an optimization of the graphics pipeline in which triangle primitives are sorted into screen-space tiles and rendered on a per-tile basis to reduce memory bandwidth and energy consumption.

[1] Additional detailed 3D graphics background information is provided in the appendix.

2.2 Vulkan Graphics API

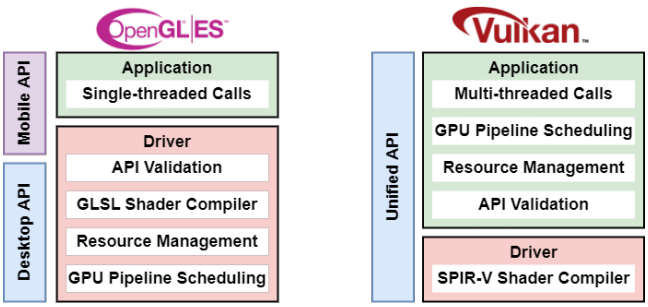


Figure 2: OpenGL vs. Vulkan Graphics APIs.

Graphics applications communicate with the GPU hardware via a standard API that translates rendering calls into hardware commands. OpenGL and OpenGL ES [15] were the traditional 3D APIs for desktop and mobile devices, respectively. Recently, the Vulkan graphics API [16] was proposed as a unified desktop and mobile API that leverages the scalability and diversity of both modern CPU and GPU architectures by delegating synchronization and performance-critical components of the runtime API to the application. Figure 2 contrasts the OpenGL software stack compared to Vulkan, highlighting the driver overhead and delegation of responsibilities. Vulkan has become today's standard API for graphics, and novel technologies, such as ray tracing, are only implemented there, which raises the importance of supporting this new API for architecture research.

2.3 RISC-V GPU Extension

The following instructions are essential to extend the RISC-V ISA to support GPGPUs [54]: **vx_wspawn** activates a number of wavefronts at a specific program's program counter (PC) value, enabling multiple instances of that program to execute independently. **vx_tmc** activates or deactivates threads within a wavefront via a thread mask register. **vx_split** and **vx_join** handle control divergence. The **vx_split** instruction pushes information about the current state of the thread mask and the branch predication result for all threads into a hardware immediate postdominator (IPDOM) stack, and the **vx_join** instruction pops this out during convergence. **vx_bar** synchronizes wavefront execution at barrier locations. **vx_tex** is used for texture lookup. The **vx_tmc** and **vx_tex** instructions are controlled by control and status registers (CSRs).

3 SOFTWARE AND HARDWARE BOUNDARIES

3.1 Graphics Hardware Classification

The GPU graphics pipeline is very complex and consists of several components that are chained together to execute graphics workloads. Depending on the implementation, graphics components are implemented as fixed-function hardware, as programmable GPU shaders, or even as emulation software on the CPU. Decisions on how to implement these various components have not only a considerable impact on the performance and die area of the accelerator, but also programmability. Table 1 shows a taxonomy of commercial

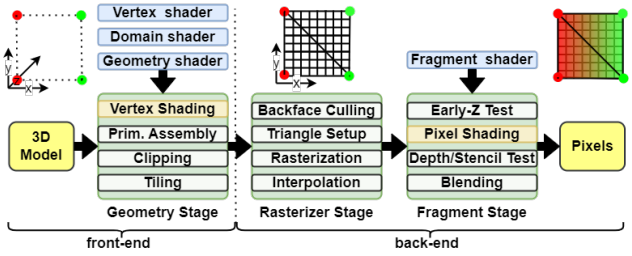


Figure 3: Baseline 3D graphics pipeline stages, with fixed-function units (white) and programmable shaders (orange).

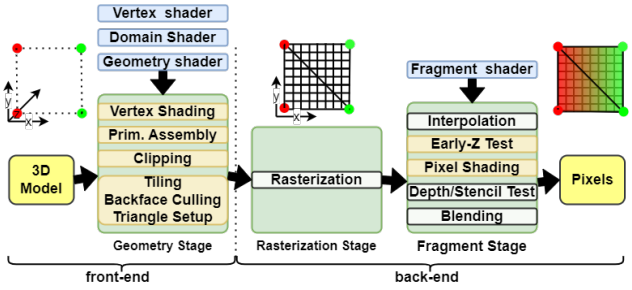


Figure 4: Skybox 3D graphics pipeline stages, with fixed-function units (white) and programmable shaders (orange).

GPUs, highlighting the implementation model of their graphics pipeline components.[2] Early GPUs [1] [58] had their geometry front-end executing on CPU to reduce the cost of the die. In the decade that followed, several pipeline components migrated from fixed-function units to execute as a programmable shader. Similar to modern GPUs, Skybox implements unified shader cores with programmable fragments and compute pipelines. Similar to Larrabee [51], Skybox's graphics pipeline is tile-based, and its geometry front-end executes on the programmable shader cores.

Figure 4 illustrates the Skybox 3D graphics pipeline, highlighting its programmable software versus fixed-function hardware boundary. The fixed-function units inside the GPU implement rasterization, interpolation, depth/stencil testing, and blending. The programmable shader cores handle early-Z testing and fragment shading. Exploring the design space of software versus hardware rendering was a key motivation for the Skybox hybrid graphics architecture.

3.2 Hybrid Rasterization

The traditional rasterization pipeline performs primitive setup and interpolation to generate per-fragment color values.

3.2.1 Triangle Setup. Skybox's tiling implementation uses the edge equations (Appendix A.1) of each triangle primitive to perform the coverage test [51] [38] [32] and determine which tiles are discarded. Skybox implements 2D homogeneous rasterization [45] [38] which also uses edge equations for the triangle setup. Because of this, we merged the tiling and triangle setup on the programmable shader cores. This design presents the following advantages: 1) We avoid

[2] More GPU architectures are compared in Appendix.

Table 1: GPU Graphics Pipeline Summary (An extended version is at Appendix Table 7) (FF: Fixed-Function)

GPU	Year	APIs	Rendering Model	ISA	Geometry Stage			
					Vertex Fetch	Vertex Shading	Triangle Setup	Tiling
Larrabee [51]	2010	OpenGL 3.2	Tile-based	X86	GPU	GPU	GPU	GPU
GeForce30 [65]	2022	Vulkan 1.3	Immediate	Proprietary	FF	GPU	FF	N/A
SkyBox	this paper	Vulkan 1.0	Tile-based	RISC-V	**GPU**	GPU	**GPU**	**GPU**

GPU	Fragment Stage											
	Raster	Early Z	Inter-polation	Pixel Shader	Texture Sampler	Late Z	Blending	Unified Shader	Multi-Sampling	Compute Shader	Ray Tracing	Tensor
Larrabee	CPU	CPU	CPU	CPU	FF	CPU	CPU	Yes	Yes	Yes	No	No
GeForce30	FF	FF	FF	GPU	FF	FF	FF	Yes	Yes	Yes	Yes	Yes
SkyBox	FF	**GPU**	FF	GPU	FF	FF	FF	Yes	Yes	Yes	No	No

having to re-compute the edge equation of each primitive during rasterization. The edge equation algorithm involves 18 multiply-add arithmetic operations plus 3 for the determinant. 2) We have some savings in memory bandwidth when storing the edge equation parameters (*E0, E1, E2*) instead of the vertex coordinates (*X, Y, Z, W*) for rasterization. 3) We can also move backface culling into the front-end to execute together with the triangle setup using the sign of the edge equation determinant (Eq. (1) in Appendix A.1) to reject culled primitives, therefore saving additional memory bandwidth.

† **Design decision:** *Merge tiling, triangle setup, and culling to the programmable shader front-end*
✻ **Reason**: Avoid recompute, improve memory bandwidth

3.2.2 Attribute Interpolation. To perform interpolation, the typical GPU rasterizer will fetch each attribute from memory, apply the interpolation, then store the result into a register file for the fragment shader to access. Skybox moved attribute interpolation from the rasterizer stage into the fragment stage to execute together with the fragment shader for the following reasons: 1) The area cost of the attributes register file is significant: $64 \times 4 \times 4 = 1024$ bytes for each pixel. On a multi-threaded platform where each thread operates on a pixel, this cost is multiplied by the total active threads on the system, occupying a much larger area than the general register file. 2) The interpolation performance is mainly driven by 2 operations: the memory transaction to load the attribute, and the actual computation that involves several multiply-add calls. The memory transfer is the main bottleneck, regardless of if it is done inside the rasterizer or not and using the available shader cache. Although the interpolation takes place in the fragment stage, Skybox implements a custom integer multiply-add (imadd) instruction to speed up the interpolation of fixed-point attributes that are prevalent in the majority of applications, as discussed in section 4.2.

† **Design decision:** *Move attribute interpolation to the fragment stage*
✻ **Reason**: Reduce register requirement

3.2.3 Early-Z Optimization. Early-Z Optimization is one of the fixed-functions in the graphics fragment processing stage, where the pixel depth value is tested against the depth buffer to check for visibility. Rejecting hidden pixels at this stage allows skipping the fragment shader invocation. Skybox performs early-Z optimization inside the programmable shader for the following reasons: 1) Doing early-Z outside the shader requires adding special hardware just for interpolating the Z-coordinates. 2) Early-Z is an optimization that is not always enabled because some applications compute the depth value inside the fragment shader. 3) When multisample anti-aliasing (MSAA) is enabled, performing early-Z interpolation in fixed-function adds storage cost to the pixel attributes for the per-sample generated depth values.

† **Design decision:** *Convert early-Z optimization from fixed-function units to the programmable shader*
✻ **Reason**: Remove storage and additional hardware cost

3.2.4 Render Output. Also called output merger, this stage of the graphics pipeline integrates the depth/stencil testing and blending fixed-function units. Skybox implements these components in fixed-function for the following reasons: 1) Their area cost can be controlled if the supported destination formats are kept to a minimum. Skybox supports a single 32-bit depth/stencil format, S8D24, with a 24-bit depth value and 8-bit stencil value, and a single 32-bit color buffer format, A8R8G8B8, with 8 bits each for the alpha, red, green, and blue values. 2) Performing the render output stage in software is actually suboptimal in terms of performance, as we will show in section 7.4.

† **Design decision:** *Keep depth/stencil and blending as fixed-function units*
✻ **Reason**: Benefit exceeds the hardware cost

4 HARDWARE IMPLEMENTATION

4.1 Skybox Microarchitecture

Figure 5 shows the overview of the Skybox microarchitecture. At the top level, the processor implements a command processor for

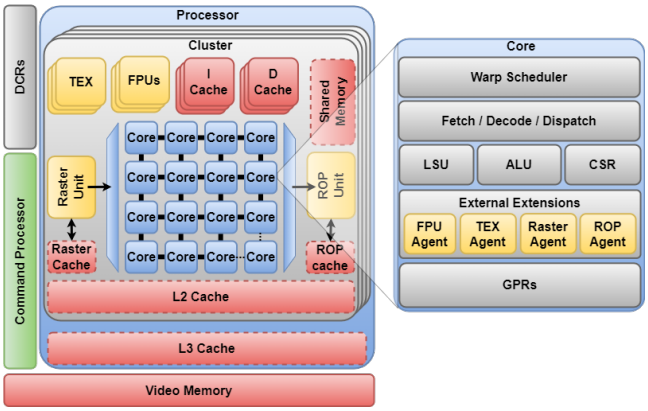


Figure 5: Skybox microarchitecture.

interfacing with the host CPU and local video memory, a device configuration register file that stores the configuration parameters of the fixed-function units, and a configurable number of clusters, each containing a configurable number of cores. We modified the baseline microarchitecture [54] to move the texture units, floating point units, instruction cache, data cache, and shared memory from individual cores to the clusters of cores to control which hardware resources are shared among cores, thus controlling the area utilization. For instance, a configuration may have 8-to-2 sharing ratio between cores and texture units (TEX) within a cluster if texture performance is not critical to the desired target application domain. Design flexibility applies to all other resources inside the cluster, including the L1 caches and shared memory. Likewise, the ability to disable any of these components is also supported, enabling a wide range of configurations when doing design space exploration.

Inside each core, we introduced lightweight hardware components that we call "agents", which are responsible for servicing requests going to or coming from external corresponding fixed-function units. A fixed-function agent will often integrate local storage to hold per-thread registers that enable data transfer between its corresponding fixed-function unit and the programmable shader via RISC-V CSR [57] instructions.

Skybox graphics extension introduces a configurable number of rasterizer (Raster) units and render output (ROP) units at the cluster level, with their respective agents in each core.

4.2 ISA Extensions

Table 2 summarizes the proposed ISA extensions.[3]

Table 2: Proposed Skybox ISA extension.

Instructions	Operands	Description
vx_rast		Raster Load
vx_rop	%pos,%face, %color, %depth	ROP Store
vx_imadd	%a, %b, %c, %shift	Interpolate

[3]In addition to these three instructions, we also extend **vx_tmc** to **vx_pred** and discuss it in Section 5.2.

vx_rast queries the rasterizer to request pixel stamps, which are discussed in section 4.3. The call is forwarded to the raster agent, which will fetch the next pixel stamp generated by the rasterizer and save them into its local CSRs for the fragment shader to access.

vx_rop submits the fragment shader result (position, face, color, and depth data) for each pixel to the ROP agent for it to be forwarded to the ROP unit. That result will eventually be written by the ROP unit to the destination render target at the provided pixel position. The face operand is a boolean value that indicates if the primitive is front-facing or back-facing during stencil operation. This instruction uses four source operands, but the RISC-V ISA only supports 3 operands at most, so we combined the position and face argument into a single register operand.

vx_imadd extends the RISC-V ISA with an integer multiply-add instruction. We added a 4th immediate operand to apply an optional left shift before the addition to support fixed-point multiplication. A *shift* value of zero enables the integer operating mode. This instruction is used to accelerate the interpolation of fixed-point attributes.

4.3 Rasterizer Unit Microarchitecture

The main role of the rasterizer unit is to convert the triangle primitives generated from the front-end into screen-space pixel quads. Figure 6 shows its architecture overview.

Data Structure Layout: The tile data is stored in two different data structures: a tile buffer ❶, and a primitive buffer ❷. The tile buffer contains a consecutive list of tile data consisting of a primitive count (Prim count) that denotes the number of triangles covered by the tile followed by their indices (pid) in the primitive buffer. The primitive buffer is an indexable array of the primitive data consisting of edge equation coefficients and interpolation attributes.

Raster Cache: The tile binning process performed on the GPU is optimized for sorting the tile data such that tiles that share the same primitives are placed closer, leading to a higher hit rate in the cache.

Raster Memory Unit (RMU): The memory unit ❸ sequentially accesses the tile buffer to fetch the tile data and accesses the primitive buffer to fetch the primitives using a finite-state machine (FSM). The FSM generates memory addresses until all tiles with their associated primitives are fetched.

Raster Slice (RS): The raster slice ❹ recursively traverses each triangle primitive from the RMU by recursively subdividing the tile (TE) ❺ to generate 2x2 quads of pixel stamps that are covered by that primitive. The primitive edge equation coefficients are used to test if the primitive is covered by the current tile, and are updated every time the tile is subdivided. This traversal is implemented using an FSM and takes several cycles to complete. Typical rasterizer implementations [8] will perform this traversal until it reaches quad elements. However, for Skybox, we added an optimization to the traversal by introducing a block abstraction, which consists of a 2-dimensional grouping of consecutive quads. Instead of iterating recursively until we reach each quad, the tile traversal will stop at the block boundary and statically evaluate all the quads (QE) in the block in parallel in the one cycle (BE) ❻, improving the dispatch throughput. The generated quads are pushed into an output queue ❼ to be sent to the cores. The block size is configurable to allow

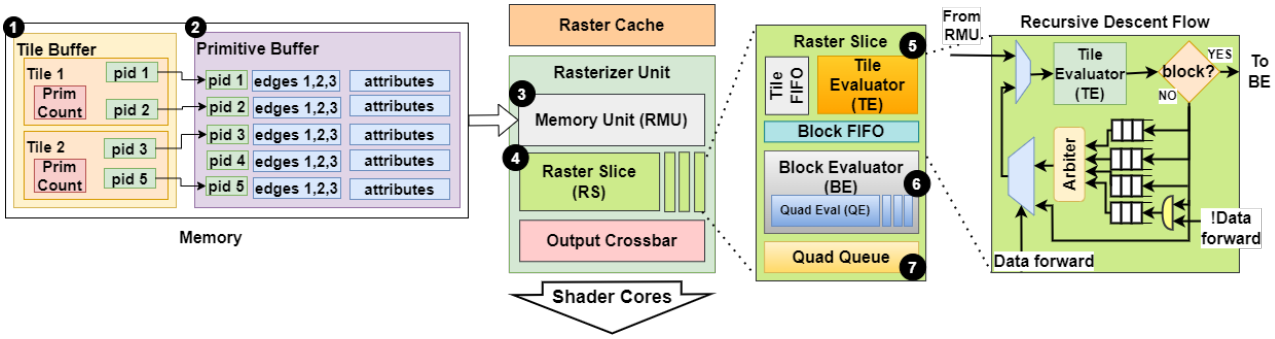


Figure 6: Rasterizer System Overview.

a trade-off between throughput and area cost. The tile size is also configurable and directly controls the number of recursive-descend iterations to reach the block. In our evaluation, we found that a tile size of 32 and a block size of 4 was the sweet spot.

```
void shader_function(kernel_arg_t* __UNIFORM__ arg) {
  for (;;) {
    // fetch next rasterized quad
    auto __DIVERGENT__ status = vx_rast();
    if (0 == status) // done?
      return;
    auto pid       = CSRS[PRIMITIVE_ID];
    auto primitive = arg->prim_addr[pid];
    auto face      = primitive.face;
    auto attribs   = primitive.attribs;
    // gradients calculation
    auto bcoords   = CSRS[BARYCENTRIC_COORDS];
    auto (dx,dy)   = GRADIENTS(bcoords);
    // attributes interpolation
    auto z = INTERPOLATE(dx, dy, attribs.z);
    auto u = INTERPOLATE(dx, dy, attribs.u);
    auto v = INTERPOLATE(dx, dy, attribs.v);
    // color texturing
    auto c = TEXTURING(u,v);
    // render output
    auto pos  = CSRS[POSITION];
    auto mask = CSRS[COVERAGE_MASK];
    for (auto i : mask) {
        vx_rop(pos[i],face,c[i],z[i]);
    }
  }
}
```

Figure 7: Shader code example: each thread executes it.

4.4 Render Output Microarchitecture

Render output units (ROP) compute per-sample fragment operations in the last stage of the rendering pipeline and write the resulting color and depth to destination buffers. Each ROP slice contains a deadlock handler, a memory unit, a depth-stencil test unit, and a blending unit, as shown in Figure 8.

Memory Unit: The memory unit generates memory requests to retrieve the depth/stencil values and destination color for each incoming pixel coordinate. It utilizes the ROP cache to reduce the DRAM traffic. The depth, stencil, and color values retrieved are sent

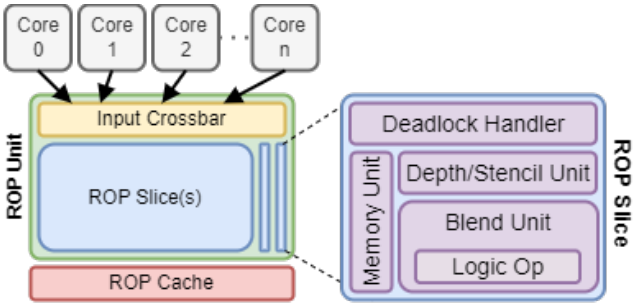


Figure 8: ROP unit.

in parallel to the depth-stencil and blend units. The memory unit is also responsible for generating write requests for the depth-stencil and blending unit outputs.

Depth-Stencil Unit: The depth test (Figure 9) compares the current fragment's depth value to its corresponding value in the depth buffer ①. If it fails, indicating that the fragment is occluded by an existing object, the fragment is discarded. The stencil test is an optional extension to the depth test. If the depth test fails, the fragment is no longer drawn, but the stencil buffer can still be affected. The stencil operation ③ is a function of the depth comparison and the stencil comparison ②. The result of the stencil operation is written to the stencil buffer.

Blender Unit: As shown in Figure 10, this unit combines the current fragment's color with the destination color stored in the framebuffer using the blend equations in Appendix A.3. The hardware schedules this operation on a two-stage pipeline, where the first stage produces the source and destination blend functions and the second stage implements the blend equation.

Deadlock Handling: The memory unit services both read and write requests to the depth, stencil, and color buffers. The deadlock unit ensures that a possible deadlock does not arise when the memory unit's request queue is populated entirely with read requests, thereby blocking the depth-stencil and blend units' write requests to memory.

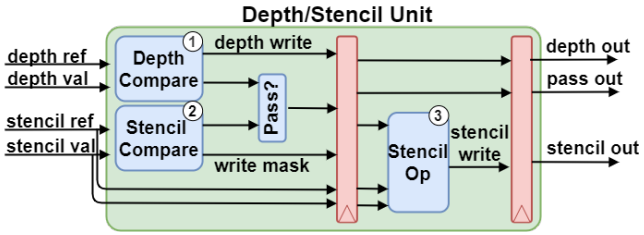


Figure 9: Depth/Stencil Unit.

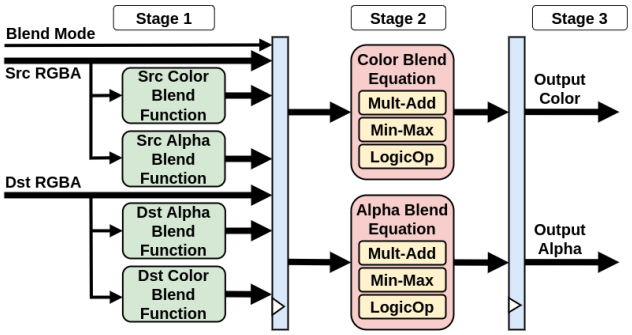


Figure 10: Blending Unit.

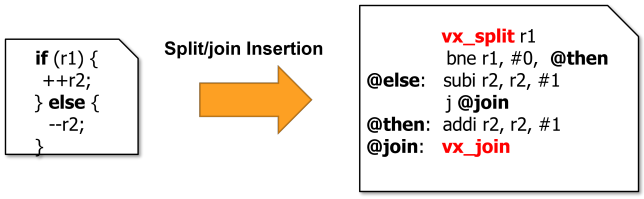


Figure 11: Divergent Branch Transformation.

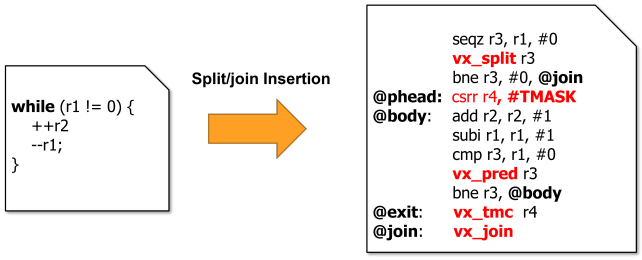


Figure 12: Divergent Loop Transformation.

5 COMPILER SUPPORT: HANDLING DIVERGENT BRANCHES

With more aggressive single-instruction, multiple-thread (SIMT) architectures on modern GPUs, as well as Vulkan [49] applications, which contain many divergent branches, it is critical to support divergent branches at compile time because manually inserting intrinsics, as done in the previous work, is not feasible. Other work [4] [48] [35] has also explored the implementation of GPU control-flow divergence at the compiler level. However, unlike other mainstream GPUs, the RISC-V GPU uses only **vx_split/vx_join** to control divergence without any predicated execution, so Skybox requires different code transformations to support branch divergence. This section describes the changes to the LLVM [33] compiler to support control-flow divergence.

5.1 If-Then-Else Transformation

To automate the insertion of **vx_split/vx_join** for basic if-then-else divergent control-flows (Figure 11), we run the Nearest-Common-Post-Dominator analysis in the LLVM compiler to identity the IPDOM basic block where the branch converges. We insert the **vx_split** before the branch and the **vx_join** at the IPDOM reconvergence point. Upon the execution of **vx_split**, the hardware will push the next PC value following where the branch is located into its IPDOM stack with the inverse mask of the not-taken threads, then activate only the threads for the taken branch. When the execution reaches the **vx_join** instruction, the IPDOM stack is popped and the control flow jumps to the stored PC value where the branch is located and activates the non-taken threads using the stored mask.

5.2 Loop Transformation

Handling divergent loops (Figure 12) is a more challenging task. Because of the multiple iterations of the control-flow, we need to know when to exit the loop. Prior work [35] handles loops by tracking the remaining active threads during the iteration and exiting when that counter goes to zero. We need to capture the original thread mask prior to entering the loop so that we can restore it at the exit point. We also need a mechanism to update the active thread during loop iterations. For this, we introduced two additions to the existing ISA: 1) a TMASK CSR that returns the currently active thread, and 2) a **vx_pred** instruction that changes current active threads. This new instruction extends the existing **vx_tmc** instruction in that it activates a thread based on its source register operand value being non-zero, while **vx_tmc** works at the warp level, using only the operand of thread 0 as a mask to activate all threads. The **vx_pred** instruction is also unique in that when its source operand is zero for all threads, no change is applied. This is to ensure that we do not disable the entire wavefront and stop execution on the last iteration of the loop. We automated this transformation in LLVM by 1) capturing the current thread mask inside the loop pre-header block, 2) inserting the **vx_pred** before the loop body branch, 3) restoring the thread mask at the beginning of the loop exit block, and 4) inserting the convergence **vx_join** pair for the earlier **vx_split**. The join is inserted at the loop IPDOM.

One limitation of the split/join divergence resolution method is that the control-flow graph has to be in structured form, meaning that it should have single-entry and single-exit nodes for each region covered by split/join. For this, we added a pre-pass to the compiler to transform each function into a structured form.

5.3 Divergence Analysis

Another important addition to the compiler is the extension of the divergence analysis phase. This phase executes before split/join insertion and performs a data-flow analysis of the divergent variables in the function. The traversal is based on the assumption that any

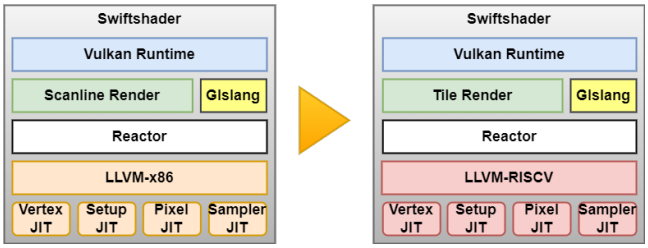


Figure 13: Original (left) & Skybox (right) Swiftshader Render.

divergent source operand of an instruction will produce a divergent result. We use the current **thread identifier** as the starting point of the data-flow analysis in the kernel and all instructions affected by it are assumed divergent. Skybox's stack is local to each thread and, as such, will only diverge if the value stored on the stack is divergent. To reduce the overhead of the divergence instrumentation, we implemented a new optimization in the compiler that exploits this feature to track uniform variables on the stack. Divergence analysis is conservative to ensure proper program execution. There are scenarios where the user may know that a value is uniform (e.g. function arguments) or divergent (e.g. custom instruction), for those cases we provide two compiler annotations: **__UNIFORM__** and **__DIVERGENT__** to guide the divergence analysis (see Figure 7).

6 SOFTWARE SUPPORT

Swiftshader [12] is a high-performance CPU-based implementation of the Vulkan graphics API. The framework uses just-in-time (JIT) compilation to speed up execution on the target architecture. Its software architecture (Figure 13) includes: 1) an implementation of the Vulkan runtime API, 2) the rendering pipeline that integrates a scanline rasterizer [27] generated at JIT time, 3) Reactor, a platform-independent JIT library with single-instruction, multiple-data (SIMD) support, that internally uses the LLVM [33] back-end to generate native binaries, and 4) Glslang [13], a front-end compiler that translates GLSL [67] shader programs to SPIR-V [29] instructions.

The following major changes were added to Swiftshader to integrate Skybox as a new rendering device:

(1) Conversion of the scanline render to a tile-based render to save memory bandwidth (section 6.1)
(2) Extension of the JIT engine for RISC-V (section 6.2)
(3) Explicit hardware mapping (section 6.3)

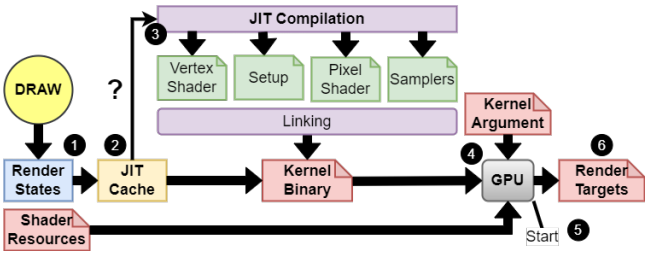


Figure 14: Skybox DrawCall Invocation.

6.1 Tile Renderer Pipeline

The original Swiftshader rasterization model was scanline-based, where during the execution of the setup JIT routine, the edge information for each horizontal scanline in the destination surface is generated. This original design does not scale well for Skybox because of the large memory bandwidth needed to transfer the scanlines to the rasterizer. We implemented a tile-based setup stage where we perform binning [41] and generate the tile buffer and primitive buffer data structures (see Figure 6) to send to the rasterizer. Figure 14 shows the execution flow taking place during a typical draw call: 1) gather pipeline render states, 2) check the JIT cache if JIT binary matching the current render states is present, 3) if JIT is needed, the JIT compilation is started to generate a new kernel, 4) offload the kernel binary with arguments and shader resources to the GPU, 5) invoke the GPU execution, and 6) gather updated render targets.

6.2 Swiftshader JIT Compiler

We made the following modifications to Swiftshader's JIT compiler stack to support RISC-V code generation for Skybox:

RISC-V JIT Support enables the compilation of graphics shaders (geometry, pixel, and samplers) to RISC-V. The JIT functions are generated as separate object files and linked into a kernel binary during the draw call.

Scalarization Support enables the JIT compiler to generate both SIMD instructions for the host CPU and scalar instructions for the GPU RISC-V architecture.

Types Alignment Support is an extension of scalarization that conditionally relaxes the memory alignment restriction on data types, whether the target architecture is the host CPU or the RISC-V GPU.

6.3 Texture Mapping

Because Skybox supports hardware texture acceleration, we need to tell the compiler when to use the hardware. We also need to consider that the texture unit doesn't support all texture formats and interpolation modes, e.g. anisotropic filtering. The compiler must provide a software fallback for texture mapping when not supported on hardware. Vulkan's VkDescriptorImageInfo provides an object encapsulation for the texture states and format used during texture mapping. We analyze the object once at creation time to identify the type of sampler code (software or hardware) to generate.

6.4 64-bit to 32-bit Domain Crossing

To support offloading the graphics kernel to the GPU, several data structures are shared between the host of the JIT compiler. These data structures are passed to the GPU kernel as arguments to provide constant resources and control states for the generated shader. These data structures are updated on the CPU and transferred during every draw call if their content changes. The challenge is that the host CPU is a 64-bit machine while Skybox is 32-bit, which creates a mismatch for embedded pointers. We flagged all domain-crossing data structures and implemented a static compilation pass that traverses those data structures' object hierarchy to generate the

32-bit equivalent types with their associated marshalling routines for data transfer.

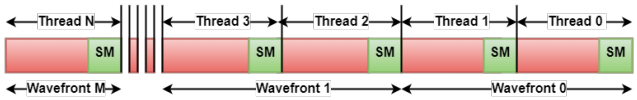


Figure 15: Skybox hybrid stack extension with shared (green) and global (red) memory.

6.5 Hybrid Stack Extension

We observed that the generated Vulkan shaders tend to be large. One reason is that Skybox is a scalar processor, and to support pixel quad execution, we have to replicate all generated instructions four times, which can cause a lot of register spill for complex shaders. This is even more challenging in a RISC-V-based GPU since each thread has a fixed number of registers (32), unlike NVIDIA's architecture. The Skybox stack originally resided on per-thread local shared memory, which is allocated at startup time. Because this local memory is expensive, supporting 8KB per thread is unpractical, as for a typical core with 4 wavefronts and 4 threads, 16 x 8KB = 128KB of local memory will be needed. As a solution, we implemented a hybrid stack extension where the stack address space for each thread is mapped to global memory, but the higher addresses of each space allocated to a thread are mapped on the local shared memory Figure 15). This provides the flexibility of resizing the local stack space on demand without recompiling the application.

7 EVALUATION

7.1 Experimental Setup

Our evaluation setup consisted of a 3.5 GHz Intel Xeon E5-1650 for the host processor. We synthesized Skybox RTL on the Intel Stratix 10 FPGA at 230 MHz. The default configuration is a 32-core system; 2 clusters of 16 cores connected to a Rasterizer and ROP units. Each core has 4 wavefronts with 4 threads each. The tests are done with a mixture of simulation (using simX, a cycle-level simulator, and RTL simulation) and ran on the FPGA. Section 7.2 shows the results of running demo programs on the FPGA.

7.2 Performance Evaluation on FPGA

We use eight 3D demos (Figure 18) from the PowerVR SDK [24] that we ported to our software library for evaluation and custom samples of our own. They provide a good mix of polygon counts and rendering features that directly exercise the major components of our graphics pipeline. The detailed descriptions of each sample are provided in Table 3, with the rightmost columns indicating if the depth/stencil and blend units are in use. The FPGA render time for each scene at Full-HD resolution is shown in Figure 16, showing good scaling with increased core count. Our 32-core configuration on Stratix 10 can have up to 16 ROP units running at 230 Mhz, producing a total pixel fill rate of 3.7 GPixels/s.

7.3 Area Cost of Synthesis

Figure 19 shows the relative area cost of the various hardware units and their caches. The caches were configured at the following sizes:

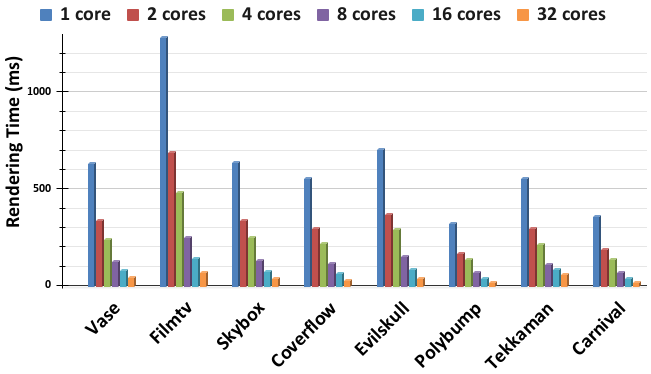


Figure 16: 3D demos rendering on FPGA at Full-HD.

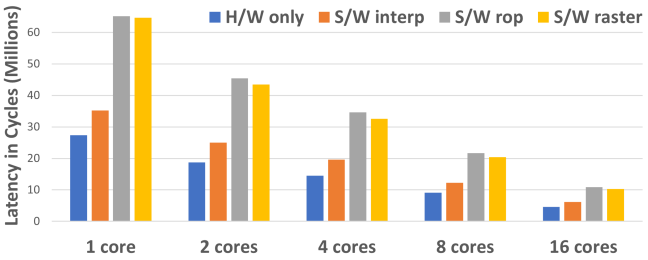


Figure 17: Software vs. Hardware Acceleration.

Table 3: Descriptions of scene demos

Name	Draws	Verts	Prims	Tex	D/S	Blend
vase	6	1198	1643	5	1	1
filmtv	16	1282	1086	11	1	1
skybox	4	704	791	9	1	1
coverflow	11	157	73	13	1	1
evilskull	7	566	699	5	1	1
polybump	3	332	276	3	1	1
tekkaman	2	2269	757	2	1	0
carnival	1	4	2	1	1	0

a 4KB rasterizer cache (RCACHE), 4KB ROP cache (OCACHE), 8KB per core texture cache (TCACHE), and 16KB per core instruction cache (ICACHE). Notably, the rasterizer and ROP additions, with their caches, account for more than a third of the combinational LUTs, BRAM, and DSP slices used by the Skybox hardware. Figure 19d shows a synthesis layout of the design on a Xilinx Alveo U280 FPGA.

7.4 Software vs Hardware Acceleration

Figure 17 shows the average software versus hardware cycle time when we selectively disable fixed-function components and use the programmable shader implementation when running the Skybox benchmark. Our software rasterizer distributes the raster tiles between the cores and wavefronts on the processor. Using Skybox baseline hardware and disabling the fixed-function rasterizer to use software rendering results in about an 129% decrease in cycles on

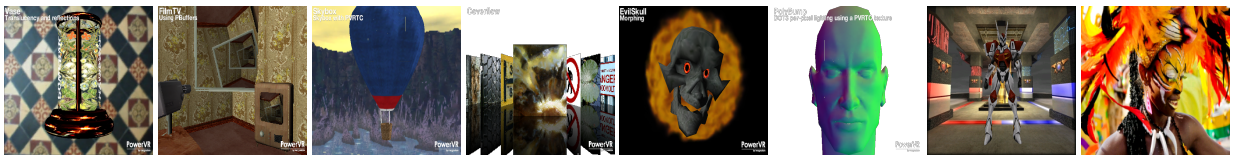
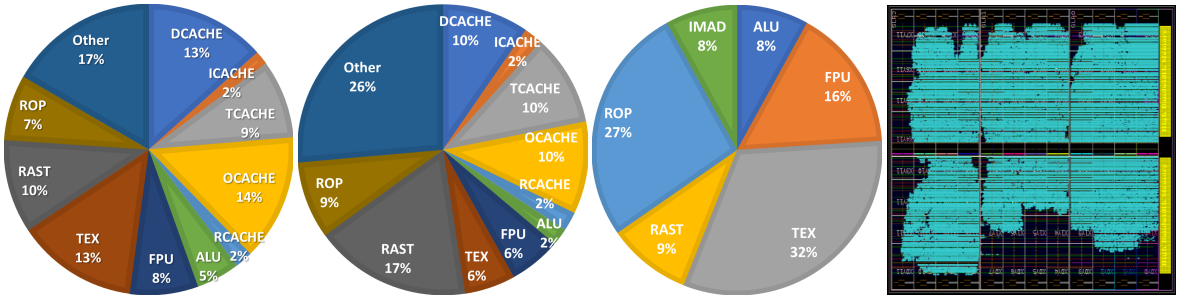


Figure 18: Snapshot of evaluated demos (Vase, Filmtv, Skybox, Coverflow, Evilskull, Polybump, Tekkaman, Carnival).



(a) LUTs **(b) BRAMs** **(c) DSPs** **(d) Layout**

Figure 19: Skybox area cost in LUTs, BRAMs, DSPs, and layout.

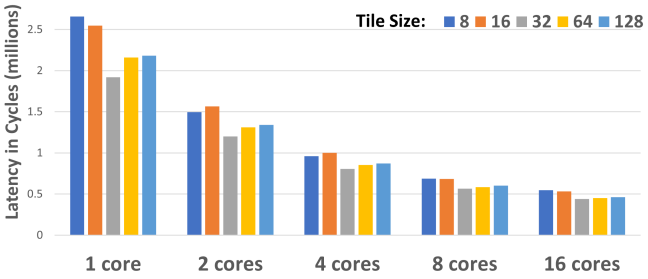


Figure 20: The effect of varying the rasterizer tile size.

average, which shows the benefit of a hardware-based rasterizer. When we instead only disable the render output fixed-function, the performance decreases by 139% on average. This gain is higher than rasterization due to its per-pixel cost. This is significant considering that the ROP is about the size of the FPU in terms of area. When we disable the custom **vx_imadd** interpolation instruction, we observe a degradation of 33% on average. Although Skybox does not have full hardware interpolation, the custom **vx_imadd** instruction introduces considerable performance improvements. These results show that all three units make effective use of the hardware. Although the difference between software interpolation and the hardware is not high, the area cost of the imadd unit is negligible, making that unit very effective as it can also be used for generic computation.

7.5 Design Space Explorations

7.5.1 Effects of the Tile size. To study the performance effect of changing the rasterizer tile size, we vary the tile size from 8x8 to 128x128 when running the Carnival benchmark with fragment shading disabled on a 512x512 render target (Figure 20). Using a small tile size increases the tile binning time and tile buffer size, which also increases the RMU bandwidth. Using a large tile size increases the number of recursive descent iterations in the rasterizer to generate the quads. So, it is beneficial to set the tile and block sizes within a few factors of each other to reduce the recursive descent cost. Increasing the block size leads to an increase in the hardware cost as it will require more QEs for parallel processing. In our evaluation, using a block size of 4 and a tile size of 32 offered the best result.

7.5.2 Effect of the Raster Cache and ROP cache. With a constant tile size (64x64) with 1 core per cluster, we have tested the effect of raster cache and ROP cache in Figure 21. Surprisingly, the results show that the benefits of RCACHE and OCACHE are very negligible. We observed that the ROP cache hit rate was low (below 20%) on average. The raster cache hit rate was above 80% on average but that had little impact on overall performance because the rasterizer throughput is output-bound due to the shader cores taking too much time to process pixel stamps (see Table 4).

Table 4: Effect of varying the number of raster slices

Cores	Slices	Instructions	Cycles	IPC	Stall %
1	1	437742	186685	2.34	10.24
1	2	437742	189052	2.32	34.37
1	4	437742	188399	2.32	45.86
4	1	444504	51890	8.57	11.88
4	2	444504	52549	8.46	32.15
4	4	444504	52051	8.54	39.10
16	1	471804	19037	24.78	1.84
16	2	471804	19208	24.56	17.91
16	4	471804	18630	25.32	35.79

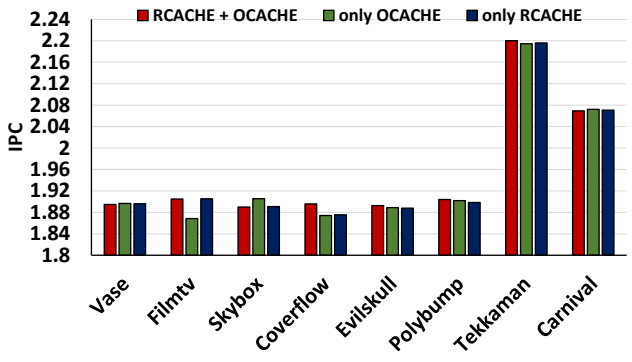


Figure 21: Rendering performance with RCACHE + OCACHE, only OCACHE, and only RCACHE on overall IPC

Table 5: Effect of varying the number of ROP slices (RTL sims)

Cores	Slices	Instructions	Cycles	IPC	Stall %
1	1	84271	156020	0.54	0
1	2	84271	153079	0.55	0
1	4	84271	154346	0.54	0
4	1	89402	174612	0.51	4
4	2	89402	179744	0.49	3
4	4	89402	174150	0.51	0
16	1	471804	192181	0.57	30
16	2	471804	192139	0.57	18
16	4	471804	189358	0.58	10

7.5.3 Increasing the number of Raster Slices. We studied the effect of increasing the number of raster slices in the rasterizer when rendering a simple quad primitive to reduce the shading overhead. The results (Table 4) show that stalling increases as the number of slices are increased. This happens as each raster slice is able to function in parallel, filling up its output queues. But, the rate of stamp requests generated from the core does not change. So, the rasterizer slices need to wait until the quad stamps in the quad queue can be dispatched. The rate of stalls is reduced with an increase in the number of cores because the rate of stamp request generation increases proportionally to the number of cores. Thus, more cores are required to fully utilize the raster slices as the rasterizer operates faster than the kernel. This evaluation guided our decision to use a single rasterizer unit per core cluster.

7.5.4 Increasing the number of ROP slices. Table 5 captures ROP stalls when the core is submitting pixels to the ROP, varying the number of cores and ROP slices when rendering a simple quad primitive to solely exercice the render output unit. The ROP unit has limited stalls only when the core count reaches about 16. This is because the shader pipeline pixels input rate is low compared to the ROP unit, so improving ROP performance doesn't help until we start having stalls around 16 cores. This evaluation guided our decision to use a single ROP with a cluster of 16 cores.

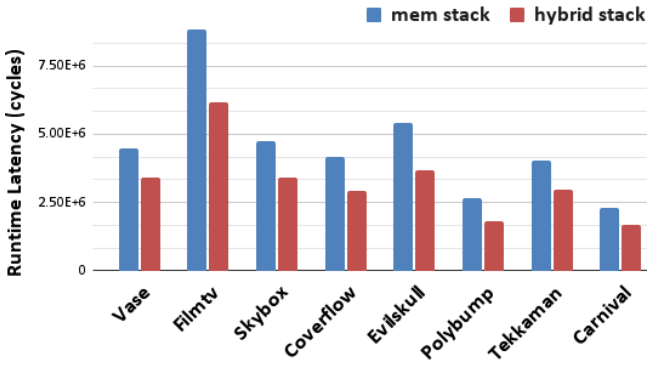


Figure 22: Hybrid stack extension performance

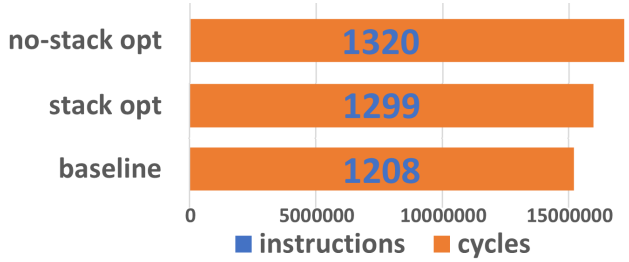


Figure 23: Divergence stack optimization

7.6 Hybrid Stack Memory Extensions

Figure 22 shows the benefits of using the hybrid stack to enable the expansion of the stack space to use global memory (Section 6.5) versus accessing global memory directly without local stack support (extension disabled). As we predicted, using the hybrid stack reduces memory accesses, improving performance by up to 22%. Filmtv has the highest number of draw calls in our suite and its shader program contains multiple texture sampling and blending operations that increase register pressure and therefore overload the stack.

7.7 Compiler Divergence Stack Optimization

Figure 23 shows a comparative evaluation of the Skybox shader program after enabling various compiler optimizations. This evaluation was conducted using a dual-core single-thread per warp configuration of the GPU. We disabled multi-threading to ensure that the program will work fine without the divergence instrumentation. The baseline bar represents the number of generated instructions and runtime latency when divergence instrumentation is disabled. The next bar is our current compiler with divergence instrumentation enabled and stack analysis optimization, while the top bar is the result when the optimization is disabled. Enabling split/join instrumentation increases code size by about 9% and our stack optimization reduces that overhead down to 7%. The corresponding slowdown overhead reduction from 13% down to 5% is more significant.

Table 6: Comparisons with related work

GPU	API	SIM	HW	GFX	Open
Skybox	Vulkan	Yes	FPGA	3D	Yes
Vulkan-sim [40]	Vulkan	Yes	No	RTX	Yes
Vortex [54]	OpenGL	Yes	FPGA	Texture	Yes
BCW-FGPU[5]	OpenGL		FPGA	3D	
ZJX-RGPU[71]	OpenCL		Chip	3D	
AccelWatch [28]	CUDA	Yes	No	No	Yes
Emerald [19]	OpenGL	Yes	No	3D	Yes
Multi2sim [55]	OpenCL	Yes	No	3D	Yes
TEAPOT [3]	OpenCL	Yes	No	3D	no
Attila [8]	OpenGL	Yes	No	3D	yes

8 RELATED WORK

Table 6 shows comparisons with other work using a GPU simulator or FPGA. Vortex [54] extends the RISC-V ISA to support GPU, however, Vortex only provides hardware texture units, whereas Skybox has hardware rasterizer and ROP units. Vortex supports only OpenGL, but Skybox also supports Vulkan, which is the latest GPU API. Recently, Vulkan-Sim [40] was released as open source for the simulation of Vulkan graphics, but it is a cycle-level simulator that executes Vulkan-traces (converted as PTX) without any RTL design. Zhou et al. demonstrate an extension to the RISC-V ISA for GPGPU tasks specifically with low power embedded systems [71]. The extension consists of a programmable vertex engine and a fixed-pipeline fragment engine to accelerate the graphics rendering pipeline. The work is demonstrated with assembly code (no software stack support) on only one GPU core.

Several researchers have implemented GPUs on FPGAs [5] [26] [30] [37] [43] [70]. Most of the work focuses on implementing fixed-graphics pipelines which provide very limited programmability and often do not support graphics APIs such as OpenGL.

NVIDIA Research has found that acceptable results are achievable with a fully software-based graphics pipeline, achieving performance within a factor of 2-8x compared to modern GPUs [31]. This research provides insight on how hardware space and rasterization performance can best be balanced to fit the design on more FPGAs with limited area.

9 DISCUSSION

9.1 Hybrid Rasterization

In our exploration of the microarchitecture design space for Skybox, we aimed to create a low-power design that could be synthesized on an FPGA while still preserving the essential capabilities of the graphics pipeline. We began by using the Intel Larrabee architecture as a baseline, given its reduced graphics die area allocation to texture units only. To prioritize our design choices, we referred to Larrabee's report on the performance breakdown of each rendering stage, which included rasterization, depth test, pixel setup, alpha blend, and pre-vertex, in that order, excluding programmable shaders. In Skybox, We choose not to implement pre-vertex in fixed function due to its low overhead and merged the pixel setup into the tile binning stage (see section 3). Skybox's evaluation revealed two key insights: 1) the high latency of the shader cores caused them to stall the rasterizer; 2) the high performance penalty of implementing the ROP stage in software. The notion that prioritizing ROP acceleration over rasterization could lead to advantages in both performance and die area cost may seem counter-intuitive. Nonetheless, this approach could offer potential benefits, and therefore warrants further exploration, particularly for embedded GPUs with a limited number of cores.

10 FUTURE WORK

Modern GPU ISAs [2] are typically 64-bit and support instructions with multiple operands, particularly graphics-centric instructions like interpolation or shader export operations that use four or more source operands. However, the RISC-V ISA instruction format only supports a maximum of three source operands. To work around this constraint, we use CSRs to pass down additional operands, but this requires a pipeline stall on each invocation. It would be beneficial to investigate a more efficient solution.

Skybox's microarchitecture only supports the scalar ISA, while the graphics shader IR is designed for SIMD operations, as it often deals with quads for pixel stamps and RGBA colors. Due to this discrepancy, the shader compiler replicates those operations on scalar machines, which quadruples the instruction count and reduces performance. To solve this issue, adding support for the RISC-V Vector ISA extension would be a viable solution.

11 CONCLUSION

With the increase in usage of mobile GPUs, partially driven by the adoption of AR and VR platforms, the need to research the graphics pipeline design is increasing. In this paper, we proposed Skybox, the first open-source hardware platform to accelerate Vulkan graphics. We evaluated Skybox on high-end Altera FPGAs (Stratix 10) and were able to synthesize and execute a 32-core (512 thread) Skybox graphics processor running at 230 Mhz, delivering a peak fill rate of 29.4 GT/s. With open hardware, we were able to propose a new hybrid GPU architecture that balances fixed-functional units and programmable units for area and performance. Open hardware and software design allows researchers to explore different graphics pipeline design options with vertical integration of the software and compiler stacks.

ACKNOWLEDGMENTS

This work was partially supported by SiliconArts. We gratefully acknowledge the support of Intel Corporation and NSF CCRI 2016701, NSF CNS 1815047 for providing FPGA resources. We would like to thank Swetha Rajagoplan, Tejaswini Anand Kumar, Sam Jijina, Jeff Young for their contribution to the Vortex project development. We also thank HPArch group members, Jeff Vetter, and the anonymous reviewers for their feedback on improving the paper.

Table 7: An extended version of GPU Graphics Pipeline Summary (Table 1 in Section 3.1) (FF: Fixed-Function)

GPU	year	APIs	Rendering Model	ISA	Geometry Stage			
					Vertex Fetch	Vertex Shading	Triangle Setup	Tiling
Voodoo [1]	1996	Glide	Immediate	Proprietary	CPU	CPU	FF	N/A
GeForce256 [58]	1999	OpenGL 1.2	Immediate	Proprietary	CPU	CPU	FF	N/A
Radeon7500 [22]	2000	OpenGL 1.3	Immediate	Proprietary	FF	FF	FF	N/A
VR-MBX [39]	2001	OpenGLES 1.0	Tile-based	Proprietary	FF	FF	FF	Yes
GeForce3 [56]	2001	OpenGL 1.3	Immediate	Proprietary	FF	FF	FF	N/A
GeForce4 [59]	2002	OpenGL 1.3	Immediate	Proprietary	FF	GPU	FF	N/A
Radeon9800 [21]	2003	OpenGL 2.0	Immediate	Proprietary	FF	GPU	FF	N/A
VR-SGX [60]	2005	OpenGLES 2.0	Tile-based	Proprietary	FF	GPU	FF	Yes
Mali-200 [6]	2007	OpenGLES 2.0	Tile-based	ARM	FF	GPU	FF	Yes
GeForce8 [62]	2008	OpenGL 3.1	Immediate	Proprietary	FF	GPU	FF	N/A
GeForce400 [63]	2010	OpenGL 3.1	Immediate	Proprietary	FF	GPU	FF	N/A
Larrabee [51]	2010	OpenGL 3.2	Tile-based	X86	GPU	GPU	GPU	GPU
GeForce20 [64]	2019	Vulkan 1.3	Immediate	Proprietary	FF	GPU	FF	N/A
GeForce30 [65]	2022	Vulkan 1.3	Immediate	Proprietary	FF	GPU	FF	N/A
SkyBox	this paper	Vulkan 1.0	Tile-based	RISC-V	**GPU**	GPU	**GPU**	**GPU**

GPU	Fragment Stage											
	Raster	Early Z	Inter-polation	Pixel Shader	Texture Sampler	Late Z	Blending	Unified Shader	Multi-Sampling	Compute Shader	Ray Tracing	Tensor
Voodoo	FF	No	FF	FF	FF	FF	FF	No	No	No	No	No
GeForce256	FF	No	FF	FF	FF	FF	FF	No	No	No	No	No
Radeon7500	FF	No	FF	FF	FF	FF	FF	No	No	No	No	No
VR-MBX	FF	No	FF	FF	FF	FF	FF	No	No	No	No	No
GeForce3	FF	FF	FF	FF	FF	FF	FF	No	No	No	No	No
GeForce4	FF	FF	FF	FF	FF	FF	FF	No	Yes	No	No	No
Radeon9800	FF	FF	FF	GPU	FF	FF	FF	No	Yes	No	No	No
VR-SGX	FF	FF	FF	GPU	FF	FF	FF	No	Yes	No	No	No
Mali-200	FF	FF	FF	GPU	FF	FF	FF	No	Yes	No	No	No
GeForce8	FF	FF	FF	GPU	FF	FF	FF	Yes	Yes	Yes	No	No
GeForce400	FF	FF	FF	GPU	FF	FF	FF	Yes	Yes	Yes	No	No
Larabeee	CPU	CPU	CPU	CPU	FF	CPU	CPU	Yes	Yes	Yes	No	No
GeForce20	FF	FF	FF	GPU	FF	FF	FF	Yes	Yes	Yes	Yes	No
GeForce30	FF	FF	FF	GPU	FF	FF	FF	Yes	Yes	Yes	Yes	Yes
SkyBox	FF	**GPU**	FF	GPU	FF	FF	FF	Yes	Yes	Yes	No	No

A APPENDIX: BACKGROUNDS ON 3D GRAPHICS PIPELINE

A.1 2D Homogeneous Rasterization

The algorithm uses the triangle edge equations to check if a pixel overlaps a triangle primitive with adjoin matrix [a,b,c] [38]:

$$E_{<1,2,3>} = [a_{<1,2,3>}, b_{<1,2,3>}, c_{<1,2,3>}] \quad (1)$$

$$E(x,y) = [a,b,c][x,y,1]^T \quad (2)$$

where E_i represents the edge between the two vertices opposite vertex v_i and E(x,y) is the edge equation value at position (x,y). Edge equations E_0, E_1, E_2 are evaluated for each pixel fragment (x,y). If all three results are negative the fragment lies inside the triangle, otherwise it is excluded.

A.2 Depth/Stencil Tests

The depth test selects visible pixels by comparing incoming fragment's depth with current depth buffer value (3). The stencil test is an extension to the depth test that enables masking-out certain portions of an image to simulate reflection or shadows (4) [50].

$$zbuf_{x,y} = \begin{cases} z, & \text{if } compare(z, zbuf_{x,y}) \\ zbuf_{x,y}, & \text{otherwise} \end{cases} \quad (3)$$

$$stencilOp = \begin{cases} depthFailOp, & \text{if } zfail \\ failOp, & \text{if } zpass \ \& \ sfail \\ passOp, & \text{if } zpass \ \& \ spass \end{cases} \quad (4)$$

where z is the depth value of fragment (x,y) and $zbuf_{x,y}$ is its corresponding depth buffer value. Stencil operations $depthFailOp$,

$failOp$, and $passOp$ are selectively executed based on the $pass$ or $fail$ status of the depth or stencil tests.

A.3 Blending Operations

This stage modulates generated pixel colors with destination buffer using either predefined blending equations (Add, Subtract, Reverse Subtract, Min, and Max) . If logical operations are used, each final pixel component's value is the result of a logical operation (and, or, xor, etc) between the source and destination pixel components [50]. The blending stage of the graphics-rendering pipeline combines the outputs of the fragment shader with their corresponding pixel colors in the framebuffer to generate the final output color of each pixel [52]. This process implements blending operations through predefined blending equations or logical operations. The blending operation to be performed is determined by the blend mode, of which there are five: Add, Subtract, Reverse Subtract, Minimum, and Maximum. An independent blend mode can be selected for both the RGB and Alpha portion of the pixel value. All five blend modes listed above take as an input the output pixel value from the fragment shader called the source value (R_s, G_s, B_s, A_s) and the current pixel value in the framebuffer called the destination value (R_d, G_d, B_d, A_d). The modes Add, Subtract, and Reverse Subtract also require two extra inputs called the source blend factor (S_r, S_g, S_b, S_a) and destination blend factor (D_r, D_g, D_b, D_a). These blend factors are set according to the selected blending function referenced in [14]. The following briefly describes each blend mode.

$$R = f(R_s * S_r, R_d * D_r) \tag{5}$$

$$G = f(G_s * S_g, G_d * D_g) \tag{6}$$

$$B = f(B_s * S_b, B_d * D_b) \tag{7}$$

$$A = f(A_s * S_a, A_d * D_a) \tag{8}$$

$$where f = Add|Sub|RevSub|Min|Max \tag{9}$$

If logical operations are used instead of blending operations, blending operations will be disabled entirely. Each final pixel component's value is then the result of a logical operation (and, or, xor, etc) between the source and destination pixel components.

A.4 Multisample Anti-aliasing

Anti-aliasing is the process of reducing the jagged edges that generally exist in the output image after rasterization. Those artifacts are reduced using various methods of super-sampling where the image is rendered at a higher resolution and then down-sampled to produce a smoother result. MSAA [25] is an optimization of super-sampling where rasterization is performed at a higher resolution and the coverage weights from those fragments are factored with the output color from only a single invocation of the fragment shader.

REFERENCES

[1] Amer Ajami. 1996. 3dfx Open Sources Glide". GameSpot. https://archive.org/details/sim_electronic-news_1995-11-06_41_2090/page/n85/mode/2up.

[2] AMD. [n. d.]. RDNA 1.0 Instruction Set Architecture. https://developer.amd.com/wp-content/resources/RDNA_Shader_ISA.pdf.

[3] Jose-Maria Arnau, Joan-Manuel Parcerisa, and Polychronis Xekalakis. 2013. TEAPOT: A Toolset for Evaluating Performance, Power and Image Quality on Mobile Graphics Systems. In *Proceedings of the 27th International ACM Conference on International Conference on Supercomputing* (Eugene, Oregon, USA) *(ICS '13)*. 37–46.

[4] Helge Bahmann, Nico Reissmann, Magnus Jahre, and Jan Christian Meyer. 2015. Perfect reconstructability of control flow from demand dependence graphs. *ACM Transactions on Architecture and Code Optimization (TACO)* 11, 4 (2015), 1–25.

[5] Alexander E. Beasley, C. T. Clarke, and R. J. Watson. 2020. An OpenGL Compliant Hardware Implementation of a Graphic Processing Unit Using Field Programmable Gate Array -System on Chip Technology. *ACM Transactions on Reconfigurable Technology and Systems* 14, 1 (2020), 2:1–2:24. https://doi.org/10.1145/3410357

[6] Ian Bratt. 2015. The arm® mali-t880 mobile gpu. In *2015 IEEE Hot Chips 27 Symposium (HCS)*. IEEE, 1–27.

[7] John Burgess. 2020. Rtx on-the nvidia turing gpu. *IEEE Micro* 40, 2 (2020), 36–44.

[8] Victor Moya Del Barrio, Carlos González, Jordi Roca, Agustín Fernández, and E Espasa. 2006. ATTILA: a cycle-level execution-driven simulator for modern GPU architectures. In *2006 IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE, 231–241.

[9] Mark Dokter, Jozef Hladky, Mathias Parger, Dieter Schmalstieg, Hans-Peter Seidel, and Markus Steinberger. 2019. Hierarchical rasterization of curved primitives for vector graphics rendering on the GPU. In *Computer Graphics Forum*, Vol. 38. Wiley Online Library, 93–103.

[10] H. Esmaeilzadeh, E. Blem, R. S. Amant, K. Sankaralingam, and D. Burger. 2011. Dark silicon and the end of multicore scaling. In *2011 38th Annual International Symposium on Computer Architecture (ISCA)*. 365–376.

[11] Francisco Ganacim, Rodolfo S. Lima, Luiz Henrique de Figueiredo, and Diego Nehab. 2014. Massively-Parallel Vector Graphics. *ACM Trans. Graph.* 33, 6, Article 229 (nov 2014), 14 pages. https://doi.org/10.1145/2661229.2661274

[12] Google. [n. d.]. SwiftShader. https://github.com/google/swiftshader/.

[13] Chronos Group. 2009. Glslang. https://github.com/KhronosGroup/glslang/.

[14] Khronos Group. 2003. OpenGL ES API. https://www.khronos.org/opengles/.

[15] Khronos Group. 2016. OpenGL API. https://opengl.org/.

[16] Khronos Group. 2016. Vulkan API. https://www.khronos.org/vulkan/.

[17] OpenHW Group. [n. d.]. OpenHW Group - Proven processor IP. https://www.openhwgroup.org.

[18] Ayub A Gubran and Tor M Aamodt. 2019. Emerald: graphics modeling for SoC systems. In *Proceedings of the 46th International Symposium on Computer Architecture*. 169–182.

[19] Ayub A Gubran and Tor M Aamodt. 2019. Emerald: graphics modeling for SoC systems. In *Proceedings of the 46th International Symposium on Computer Architecture*. 169–182.

[20] A. Gutierrez, B. M. Beckmann, A. Dutu, J. Gross, M. LeBeane, J. Kalamatianos, O. Kayiran, M. Poremba, B. Potter, S. Puthoor, M. D. Sinclair, M. Wyse, J. Yin, X. Zhang, A. Jain, and T. Rogers. 2018. Lost in Abstraction: Pitfalls of Analyzing GPUs at the Intermediate Language Level. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 608–619. https://doi.org/10.1109/HPCA.2018.00058

[21] Hilbert Hagedoorn. [n. d.]. ATI Radeon 9800 XT review. https://www.guru3d.com/articles-pages/ati-radeon-9800-xt-review,1.html.

[22] HEXUS. [n. d.]. Review:ATI Radeon 7500. https://hexus.net/tech/reviews/graphics/237-ati-radeon-7500/.

[23] Kai Hormann and Marco Tarini. 2004. A quadrilateral rendering primitive. In *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware*. 7–14.

[24] Imagination. [n. d.]. PowerVR SDKn. https://developer.imaginationtech.com/powervr-sdk/.

[25] Jorge Jimenez, Diego Gutierrez, Jason Yang, Alexander Reshetov, Pete Demoreuille, Tobias Berghoff, Cedric Perthuis, Henry Yu, Morgan McGuire, Timothy Lottes, Hugh Malan, Emil Persson, Dmitry Andreev, and Tiago Sousa. 2011. Filtering Approaches for Real-Time Anti-Aliasing. In *ACM SIGGRAPH 2011 Courses* (Vancouver, British Columbia, Canada) *(SIGGRAPH '11)*. Association for Computing Machinery, New York, NY, USA, Article 6, 329 pages. https://doi.org/10.1145/2037636.2037642

[26] Peng-Fei Jin, Shao-Jun Yao, Dong-Xiao Li, Liang-Hao Wang, and Ming Zhang. 2012. Real-time multi-view rendering based on FPGA. In *International Conference on Systems and Informatics (ICSAI2012)*. 1981–1984. https://doi.org/10.1109/ICSAI.2012.6223438

[27] Kiia Kallio. 2007. Scanline edge-flag algorithm for antialiasing. (2007).

[28] Vijay Kandiah, Scott Peverelle, Mahmoud Khairy, Junrui Pan, Amogh Manjunath, Timothy G. Rogers, Tor M. Aamodt, and Nikos Hardavellas. 2021. AccelWattch: A Power Modeling Framework for Modern GPUs. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. 738–753. https://doi.org/10.1145/3466752.3480063

[29] John Kessenich, Boaz Ouriel, and Raun Krisch. 2018. SPIR-V Specification. *Khronos Group* 3 (2018).

[30] Kyungsu Kim, Hoosung-Lee, Seonghyun Cho, and Seongmo Park. 2008. Implementation of 3D graphics accelerator using full pipeline scheme on FPGA. In *2008 International SoC Design Conference*, Vol. 02. II–97–II–100. https://doi.org/10.1109/SOCDC.2008.4815693

[31] Samuli Laine and Tero Karras. [n. d.]. High-Performance Software Rasterization on GPUs. In *Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics - HPG '11* (Vancouver, British Columbia, Canada, 2011). ACM Press, 79. https://doi.org/10.1145/2018323.2018337

[32] Samuli Laine and Tero Karras. 2011. High-Performance Software Rasterization on GPUs. In *Eurographics/ ACM SIGGRAPH Symposium on High Performance Graphics*, Carsten Dachsbacher, William Mark, and Jacopo Pantaleoni (Eds.). ACM. https://doi.org/10.1145/2018323.2018337

[33] C. Lattner and V. Adve. 2004. LLVM: a compilation framework for lifelong program analysis amp; transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.* 75–86. https://doi.org/10.1109/CGO.2004.1281665

[34] Won-Jong Lee, Youngsam Shin, Jaedon Lee, Jin-Woo Kim, Jae-Ho Nah, Seokyoon Jung, Shihwa Lee, Hyun-Sang Park, and Tack-Don Han. 2013. SGRT: A Mobile GPU Architecture for Real-Time Ray Tracing. In *Proceedings of the 5th High-Performance Graphics Conference* (Anaheim, California) *(HPG '13)*. Association for Computing Machinery, New York, NY, USA, 109–119. https://doi.org/10.1145/2492045.2492057

[35] Yunsup Lee, Vinod Grover, Ronny Krashinsky, Mark Stephenson, Stephen W Keckler, and Krste Asanovic. 2014. Exploring the design space of SPMD divergence management on data-parallel architectures. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 101–113.

[36] Erik Lindholm, Mark J Kilgard, and Henry Moreton. 2001. A user-programmable vertex engine. In *Proceedings of the 28th annual conference on Computer graphics and interactive techniques*. 149–158.

[37] Yukun Liu. 2014. A novel Mesa-based OpenGL implementation on an FPGA-based embedded system. In *International Conference on Audio, Language and Image Processing*. 78–83. https://doi.org/10.1109/ICALIP.2014.7009761

[38] Michael D McCool, Chris Wales, and Kevin Moule. 2001. Incremental and hierarchical hilbert order edge equation polygon rasterizatione. In *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS workshop on Graphics hardware*. 65–72.

[39] John Metcalfe. [n. d.]. PowerVR MBX - Creating an IP standard for advanced mobile multimedia. https://www.design-reuse.com/articles/10055/powervr-mbx-creating-an-ip-standard-for-advanced-mobile-multimedia.html.

[40] Saed Mohammadreza, Hsi Chou Yuan, Liu Lufei, Nowicki Tyler, and Aamodt Tor. 2022. A GPU Architecture Simulator for Ray Tracing. *IEEE/ACM International Symposium on Microarchitecture* 55 (2022).

[41] S. Molnar, M. Cox, D. Ellsworth, and H. Fuchs. 1994. A sorting classification of parallel rendering. *IEEE Computer Graphics and Applications* 14, 4 (July 1994), 23–32. https://doi.org/10.1109/38.291528

[42] John Nickolls and William J Dally. 2010. The GPU computing era. *IEEE micro* 30, 2 (2010), 56–69.

[43] M.A. Nuno-Maganda and M.O. Arias-Estrada. 2005. Real-time FPGA-based architecture for bicubic interpolation: an application for digital image scaling. In *2005 International Conference on Reconfigurable Computing and FPGAs (ReConFig'05)*. 8 pp.–1. https://doi.org/10.1109/RECONFIG.2005.34

[44] NVIDIA. [n. d.]. NVIDIA AMPERE GA102 GPU ARCHITECTURE. https://images.nvidia.com/aem-dam/en-zz/Solutions/geforce/ampere/pdf/NVIDIA-ampere-GA102-GPU-Architecture-Whitepaper-V1.pdf.

[45] Marc Olano and Trey Greer. 1997. Triangle scan conversion using 2D homogeneous coordinates. In *Proceedings of the ACM SIGGRAPH/EUROGRAPHICS workshop on Graphics hardware*. 89–95.

[46] Hanspeter Pfister, Matthias Zwicker, Jeroen Van Baar, and Markus Gross. 2000. Surfels: Surface elements as rendering primitives. In *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*. 335–342.

[47] Jason Power, Joel Hestness, Marc S Orr, Mark D Hill, and David A Wood. 2014. gem5-gpu: A heterogeneous cpu-gpu simulator. *IEEE Computer Architecture Letters* 14, 1 (2014), 34–36.

[48] Nico Reissmann, Thomas L Falch, Benjamin A Bjørnseth, Helge Bahmann, Jan Christian Meyer, and Magnus Jahre. 2016. Efficient control flow restructuring for GPUs. In *2016 International Conference on High Performance Computing & Simulation (HPCS)*. IEEE, 48–57.

[49] Mathias Schott. 2016. Vulkan Multi-Threading. https://developer.nvidia.com/sites/default/files/akamai/gameworks/blog/munich/mschott_vulkan_multi_threading.pdf.

[50] M Segal and K Akeley. 2017. The OpenGL Graphics System: A Specification (Version 4.0 (Compatibility Profile) March 11, 2010). *URL: https://khronos. org/registry/OpenGL/specs/gl/glspec40. compatibility. pdf (accessed 02.04. 2017)* (2017).

[51] Larry Seiler, Doug Carmean, Eric Sprangle, Tom Forsyth, Pradeep Dubey, Stephen Junkins, Adam Lake, Robert Cavin, Roger Espasa, Ed Grochowski, Toni Juan, Michael Abrash, Jeremy Sugerman, and Pat Hanrahan. 2009. Larrabee: A Many-Core x86 Architecture for Visual Computing. *IEEE Micro* 29, 1 (2009), 10–21. https://doi.org/10.1109/MM.2009.9

[52] Graham Sellers and John Kessenich. 2016. *Vulkan programming guide: The official guide to learning vulkan.* Addison-Wesley Professional.

[53] Rys Sommefeldt. 2015. A look at the PowerVR graphics architecture: Tile-based rendering. https://www.imgtec.com/blog/a-look-at-the-powervr-graphics-architecture-tile-based-rendering.

[54] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. 2021. Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 754–766.

[55] R. Ubal, B. Jang, P. Mistry, D. Schaa, and D. Kaeli. 2012. Multi2Sim: A simulation framework for CPU-GPU computing. In *2012 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 335–344.

[56] SCOTT WASSON. 2001. NVIDIA's GeForce3 graphics processor. https://techreport.com/review/2515/nvidias-geforce3-graphics-processor/.

[57] Andrew Waterman, Yunsup Lee, David A Patterson, and Krste Asanovi. 2014. *The RISC-V Instruction Set Manual. Volume 1: User-Level ISA, Version 2.0.* Technical Report. EECS Department, UC Berkeley.

[58] Wikipedia. [n. d.]. GeForce 256. https://en.wikipedia.org/wiki/GeForce_256.

[59] Wikipedia. 2002. GeForce 4 series. https://en.wikipedia.org/wiki/GeForce_4_series.

[60] Wikipedia. 2005. PowerVR. https://en.wikipedia.org/wiki/PowerVR.

[61] Wikipedia. 2007. Mali (GPU). https://en.wikipedia.org/wiki/Mali_(GPU).

[62] Wikipedia. 2008. GeForce 8 series. https://en.wikipedia.org/wiki/GeForce_8_series.

[63] Wikipedia. 2010. GeForce 400 series. https://en.wikipedia.org/wiki/GeForce_400_series.

[64] Wikipedia. 2019. GeForce 20 series. https://en.wikipedia.org/wiki/GeForce_20_series.

[65] Wikipedia. 2022. GeForce 30 series. https://en.wikipedia.org/wiki/GeForce_30_series.

[66] Global New Wire. [n. d.]. Think Silicon to Unveil Industry's First RISC-V 3D GPU at Embedded World 2022. https://www.globenewswire.com/news-release/2022/06/15/2462749/0/en/Think-Silicon-to-Unveil-Industry-s-First-RISC-V-3D-GPU-at-Embedded-World-2022.html.

[67] David Wolff. 2018. *OpenGL 4 Shading Language Cookbook: Build high-quality, real-time 3D graphics with OpenGL 4.6, GLSL 4.6 and C++ 17.* Packt Publishing Ltd.

[68] Chris Wyman and Adam Marrs. 2019. Introduction to DirectX raytracing. In *Ray Tracing Gems*. Springer, 21–47.

[69] Juwon Yun, Jinyoung Lee, Cheong Ghil Kim, Yeong-Kyu Lim, Jae-Ho Nah, Youngsik Kim, and Woo-Chan Park. 2018. A novel performance prediction model for mobile GPUs. *IEEE Access* 6 (2018), 16235–16245.

[70] P. Zemcik, A. Herout, L. Crha, O. Fucik, and P. Tupec. 2004. Particle rendering engine in DSP and FPGA. In *11th IEEE International Conference and Workshop on the Engineering of Computer-Based Systems*. 361–368. https://doi.org/10.1109/ECBS.2004.1316720

[71] Yuzhi Zhou, Xi Jin, and Tian Xiang. [n. d.]. RISC-V Graphics Rendering Instruction Set Extensions for Embedded AI Chips Implementation. In *Proceedings of the 2020 2nd International Conference on Big Data Engineering and Technology* (New York, NY, USA, 2020-01-03) *(BDET 2020)*. Association for Computing Machinery, 85–88. https://doi.org/10.1145/3378904.3378926

Received 2022-10-20; accepted 2023-01-19